

高级数据结构第九章习题答案

学号：202533354 姓名：ZHOU MINGYUAN

例题 1

题干：

给定关键字集合：

{10, 20, 30, 40, 50, 60, 70}

有两种二叉搜索树结构：

结构 A：

40

/ \

20 60

/ \ / \

10 30 50 70

结构 B：

10

\

20

\

30

\

40

\

50

\

60

\

70

回答：

- (1) 在结构 A 中查找 70，最多需要比较几次？
- (2) 在结构 B 中查找 70，最多需要比较几次？
- (3) 哪一种结构更接近 $O(\log n)$ ？
- (4) 哪一种结构退化为 $O(n)$ ？
- (5) 为什么普通二叉搜索树的性能依赖树的形状？

答案：

- (1) 在结构 A 中查找 70 最多比较 3 次：40、60、70。
- (2) 在结构 B 中查找 70 最多比较 7 次。
- (3) 结构 A 更接近 $O(\log n)$ 。
- (4) 结构 B 退化为 $O(n)$ 。
- (5) 普通 BST 的查找长度取决于树高，而树高由插入顺序和最终形状决定。

解析：

二叉搜索树每比较一次就沿左或右子树继续查找。树越平衡，高度越低；若退化成链表，查找就接近顺序查找。

例题 2

题干：

某二叉搜索树在连续插入 10, 20, 30, 40, 50 后，形成了一条向右倾斜的链。

回答：

- (1) 这种树的高度是否接近 n ？
- (2) 查找最深节点时，时间复杂度接近 $O(\log n)$ 还是 $O(n)$ ？
- (3) 如果每次都把树重建成完全二叉树，查找效率会怎样？
- (4) 为什么每次重建完全二叉树不适合作为普通插入策略？
- (5) AVL 树和红黑树主要想解决什么问题？

答案：

- (1) 这种树的高度接近 n 。
- (2) 查找最深节点的时间复杂度接近 $O(n)$ 。
- (3) 若每次都重建成完全二叉树，查找效率可接近 $O(\log n)$ 。
- (4) 每次插入后重建整棵树代价太大，通常需要移动或重连大量节点。
- (5) AVL 树和红黑树主要解决普通 BST 可能退化、操作最坏时间过高的问题。

解析：

平衡搜索树通过局部旋转、平衡因子或颜色规则维持树高为 $O(\log n)$ ，避免频繁全局重建。

例题 3

题干：

设某节点 T 的左子树高度为 3，右子树高度为 1。

回答：

- (1) 节点 T 的平衡因子 $BF(T)$ 等于多少? 2
- (2) 如果左子树高度为 2, 右子树高度为 3, $BF(T)$ 等于多少? -1
- (3) AVL 树中任意节点的 BF 允许取哪些值? -1 0 1
- (4) $BF(T)=+2$ 时, 该节点是否平衡?
- (5) $BF(T)=-2$ 时, 该节点是否平衡?

答案:

- (1) $BF(T)=3-1=2$ 。
- (2) $BF(T)=2-3=-1$ 。
- (3) AVL 树中任意节点 BF 允许取 -1、0、1。
- (4) $BF(T)=+2$ 时不平衡。
- (5) $BF(T)=-2$ 时不平衡。

解析:

平衡因子定义为左子树高度减右子树高度。AVL 树要求每个节点左右子树高度差不超过 1。

例题 4

题干:

给定如下二叉搜索树:

```
30
 / \
20 40
 /
10
```

其中叶子节点高度记为 1, 空子树高度记为 0。

回答:

- (1) 节点 10 的 BF 是多少? 0
- (2) 节点 20 的 BF 是多少? 1
- (3) 节点 40 的 BF 是多少? 0
- (4) 节点 30 的 BF 是多少? 1
- (5) 该树是否是 AVL 树? 请说明理由。

答案:

- (1) 节点 10 的 BF 为 0。
- (2) 节点 20 的 BF 为 1。
- (3) 节点 40 的 BF 为 0。

(4) 节点 30 的 BF 为 1。

(5) 该树是 AVL 树，因为它满足 BST 顺序，且所有节点 BF 都在 -1、0、1 范围内。

解析：

判断 AVL 不能只看根节点，必须检查每个节点。叶子高度为 1，空子树高度为 0，因此可以自底向上计算。

例题 5

题干：

空 AVL 树中依次插入：

30, 20, 40, 10

回答：

(1) 插入 30 后是否需要旋转？

(2) 插入 20 后是否需要旋转？

(3) 插入 40 后是否需要旋转？

(4) 插入 10 后，哪些节点的 BF 发生变化？

(5) 插入 10 后整棵树是否仍是 AVL 树？

(6) AVL 插入是否每次都必须旋转？

答案：

(1) 插入 30 后不需要旋转。

(2) 插入 20 后不需要旋转。

(3) 插入 40 后不需要旋转。

(4) 插入 10 后，节点 20 和 30 的 BF 发生变化。

(5) 插入 10 后整棵树仍是 AVL 树。

(6) AVL 插入不是每次都必须旋转，只有出现 $BF=+2$ 或 $BF=-2$ 时才需要调整。

解析：

插入后要沿搜索路径向上更新高度或 BF。若所有节点仍满足平衡范围，只需更新信息，不需要旋转。

例题 6

题干：

某 AVL 树插入新节点后，最近失衡祖先节点记为 A，新插入节点记为 Y。

分别判断下列情况属于哪种失衡类型：

(1) Y 插入到 A 的左子树的左子树中。LL

(2) Y 插入到 A 的左子树的右子树中。LR

(3) Y 插入到 A 的右子树的右子树中。RR

(4) Y 插入到 A 的右子树的左子树中。RL

(5) 哪两种属于直线型失衡？

(6) 哪两种属于折线型失衡?

答案:

- (1) 插入到 A 的左子树的左子树中是 LL。
- (2) 插入到 A 的左子树的右子树中是 LR。
- (3) 插入到 A 的右子树的右子树中是 RR。
- (4) 插入到 A 的右子树的左子树中是 RL。
- (5) 直线型失衡为 LL 和 RR。
- (6) 折线型失衡为 LR 和 RL。

解析:

失衡类型由从最近失衡祖先 A 到新插入节点 Y 的方向序列决定。直线型用一次单旋，折线型通常用双旋。

例题 7

题干：

某 AVL 树局部结构如下：

30

/

20

/

10

回答：

- (1) 最近失衡节点是哪个？
- (2) 该失衡属于 LL、LR、RR、RL 中的哪一种？
- (3) 应采用单旋还是双旋？
- (4) 旋转后哪个节点成为该局部子树的新根？
- (5) 旋转后 10、20、30 的父子关系如何？

答案：

- (1) 最近失衡节点是 30。
- (2) 属于 LL 失衡。
- (3) 应采用单旋，即对 30 做右旋。
- (4) 旋转后 20 成为该局部子树的新根。
- (5) 旋转后 20 的左孩子为 10，右孩子为 30。

解析：

LL 型表示新节点插入在左孩子的左侧。右旋后，中间关键字 20 上升，仍保持二叉搜索树的中序顺序 10, 20, 30。

例题 8

题干：

某 AVL 树局部结构如下：

30

/

10

\

20

回答：

- (1) 最近失衡节点是哪个？
- (2) 该失衡属于 LL、LR、RR、RL 中的哪一种？
- (3) 应采用单旋还是双旋？
- (4) 旋转后哪个节点成为该局部子树的新根？
- (5) 旋转后 10 和 30 分别位于新根的哪一侧？
- (6) 为什么该情况不能只用一次单旋解决？

答案：

- (1) 最近失衡节点是 30。
- (2) 属于 LR 失衡。
- (3) 应采用双旋：先对 10 左旋，再对 30 右旋。
- (4) 旋转后 20 成为局部子树的新根。
- (5) 10 位于 20 的左侧，30 位于 20 的右侧。
- (6) 因为 LR 是折线型结构，一次单旋不能同时恢复高度平衡并保持正确的局部根位置。

解析：

双旋的本质是先把折线转成直线，再执行一次相反方向的单旋。最终中间关键字上升为局部根。

例题 9

题干：

设三个关键节点满足：

$$H < I < J$$

旋转后局部结构应为：

I

/ \

H J

/ \ / \

A B C D

另有四棵子树 A、B、C、D，满足：

A 中所有 key < H

H < B 中所有 key < I

I < C 中所有 key < J

D 中所有 key > J

回答：

- (1) 子树 A 应挂在 H 的左边还是右边？
- (2) 子树 B 应挂在哪儿？
- (3) 子树 C 应挂在哪儿？
- (4) 子树 D 应挂在哪儿？
- (5) 判断子树挂接位置的根本原则是什么？

答案：

- (1) 子树 A 挂在 H 的左边。
- (2) 子树 B 挂在 H 的右边。
- (3) 子树 C 挂在 J 的左边。
- (4) 子树 D 挂在 J 的右边。
- (5) 根本原则是保持二叉搜索树的中序遍历顺序不变。

解析：

旋转只能改变局部父子关系，不能破坏 key 的大小区间。 $A < H < B < I < C < J < D$ 的顺序必须保持。

例题 10

题干：

从空 AVL 树开始，依次插入：

10, 20, 30, 25

回答：

- (1) 插入 10 后的根节点是谁？
- (2) 插入 20 后是否失衡？
- (3) 插入 30 后属于哪种失衡类型？如何旋转？
- (4) 插入 25 后是否失衡？如果失衡，最近失衡祖先节点是谁？
- (5) 插入全部完成后，画出最终 AVL 树。

答案：

- (1) 插入 10 后根节点是 10。
- (2) 插入 20 后不失衡。
- (3) 插入 30 后在节点 10 处出现 RR 失衡，对 10 左旋，20 成为根。
- (4) 插入 25 后不失衡，因此不存在需要旋转的最近失衡祖先。
- (5) 最终 AVL 树：根为 20；左孩子为 10；右孩子为 30；30 的左孩子为 25。

解析：

插入 30 形成 10→20→30 的右右链，需要一次左旋。插入 25 后，根 20 的左右高度差为 1，节点 30 的 BF 为 1，仍满足 AVL 条件。

例题 11

题干：

某 AVL 树节点结构定义如下：

```
struct treeNode {  
  
    treePointer leftChild;  
  
    element data;  
  
    short int bf;  
  
    treePointer rightChild;  
  
};
```

回答：

- (1) leftChild 的作用是什么？
- (2) rightChild 的作用是什么？
- (3) data 的作用是什么？
- (4) bf 表示什么？
- (5) 与普通二叉搜索树相比，AVL 树节点结构中最关键的新增信息是什么？

答案：

- (1) leftChild 指向左子树。
- (2) rightChild 指向右子树。
- (3) data 保存节点的数据或关键字。
- (4) bf 表示该节点的平衡因子。
- (5) 与普通 BST 相比，AVL 节点最关键的新增信息是 bf。

解析：

AVL 树需要在插入和删除后快速判断是否失衡，因此节点中保存平衡因子或高度信息，以支持局部更新和旋转。

例题 12

题干：

在 AVL 插入函数中，存在参数：

```
int *unbalanced
```

插入新节点时有如下代码：

```
if (!*parent) {  
  
    *unbalanced = TRUE;  
  
    ...  
  
}
```

回答：

- (1) `!*parent` 表示什么？
- (2) 为什么在这里将 `*unbalanced` 设置为 `TRUE`？
- (3) `*unbalanced` 是否表示“整棵树已经失衡”？
- (4) `*unbalanced` 更准确地表示什么？
- (5) 为什么递归返回时需要根据 `*unbalanced` 更新祖先节点的 `bf`？

答案：

- (1) `!*parent` 表示当前指针位置为空，即递归找到了新节点应插入的 `NULL` 位置。
- (2) 设置 `*unbalanced=TRUE` 是因为插入新节点会使该子树高度增加。
- (3) `*unbalanced` 不表示“整棵树已经失衡”。
- (4) 它更准确地表示“子树高度发生增加，需要向祖先传播更新”。
- (5) 递归返回时祖先节点要根据该信号更新 `bf`，并在必要时旋转。

解析：

`unbalanced` 在代码中是高度变化传播标志。只有当更新后的 `BF` 超出允许范围时，才是真正需要旋转的 `AVL` 失衡。

例题 13

题干：

在红黑树中，常把普通二叉搜索树扩展为扩展二叉搜索树，即在每个空链接处放置特殊方形节点。

回答：

- (1) 什么是内部节点？
- (2) 什么是外部节点？
- (3) 外部节点是否属于原始二叉搜索树？
- (4) 为什么红黑树性质中要提到外部节点？
- (5) 外部节点在红黑树中通常被看成什么颜色？

答案：

- (1) 内部节点是原始二叉搜索树中保存真实 key 的节点。
- (2) 外部节点是放在空链接处的特殊 NULL 或方形节点。
- (3) 外部节点不属于原始二叉搜索树的真实记录。
- (4) 红黑树性质要提到外部节点，是为了严谨定义从根到叶子的黑高度。
- (5) 外部节点通常被看成黑色。

解析：

把空指针显式看成黑色外部节点后，红黑树的每条根到外部节点路径都能统一计算黑色节点数量。

例题 14

题干：

判断下列说法是否正确，并说明理由：

- (1) 红黑树的根节点必须是黑色。
- (2) 红黑树中可以出现红色节点的红色孩子。
- (3) 所有外部节点都被看成黑色。
- (4) 从根节点到每个外部节点的路径上，黑色节点数必须相同。
- (5) 红黑树要求每个节点左右子树高度差不超过 1。

答案：

- (1) 正确。红黑树根节点必须为黑色。
- (2) 错误。红色节点不能有红色孩子。
- (3) 正确。所有外部节点都看成黑色。
- (4) 正确。从根到每个外部节点的路径上黑色节点数必须相同。
- (5) 错误。左右子树高度差不超过 1 是 AVL 的严格要求，不是红黑树要求。

解析：

红黑树通过颜色和黑高度维持近似平衡，而不是像 AVL 那样要求每个节点左右子树高度差最多为 1。

例题 15

题干：

一棵红黑树有 n 个内部节点。

回答：

- (1) 红黑树高度最多为多少？
- (2) 红黑树是严格平衡还是近似平衡？
- (3) 红黑树查找操作的时间复杂度是多少？
- (4) 红黑树插入操作的时间复杂度是多少？
- (5) 为什么红黑树不会像普通 BST 那样轻易退化成链表？

答案：

- (1) 红黑树高度最多为 $2\log_2(n+1)$ 。
- (2) 红黑树是近似平衡，不是严格平衡。
- (3) 查找操作时间复杂度为 $O(\log n)$ 。
- (4) 插入操作时间复杂度为 $O(\log n)$ 。
- (5) 颜色规则限制了最长路径与最短路径的差距，因此不会轻易退化成链表。

解析：

红黑树保证任意路径上的黑高度一致，且红节点不能连续出现，所以最长路径不会超过最短路径的大约两倍。

例题 16

题干：

红黑树节点定义如下：

```
typedef enum {red, black} color;

typedef struct redBlack {

    element data;

    redBlackPtr leftChild;

    redBlackPtr rightChild;

    color Color;

};
```

回答：

- (1) color 枚举类型有哪些取值？
- (2) Color 字段用于保存什么？
- (3) 红黑树和普通二叉搜索树相比，多了什么关键信息？

(4) 红黑树和 AVL 树相比，一个主要通过什么维护平衡，另一个主要通过什么维护平衡？

(5) 红黑树节点中是否需要保存 bf？

答案：

- (1) color 枚举类型有 red 和 black 两个取值。
- (2) Color 字段用于保存节点颜色。
- (3) 红黑树比普通 BST 多了颜色信息。
- (4) 红黑树主要通过颜色规则维护近似平衡；AVL 树主要通过 bf 或高度差维护严格平衡。
- (5) 红黑树节点通常不需要保存 bf。

解析：

红黑树插入删除后的修复依据是节点颜色、父节点、祖父节点和叔节点关系，而不是每个节点的平衡因子。

例题 17

题干：

向一棵红黑树插入新节点 a。

回答：

- (1) 如果原树为空，新节点 a 应染成什么颜色？为什么？
- (2) 如果原树非空，新节点 a 通常先染成什么颜色？
- (3) 为什么非空树中不通常直接把新节点染成黑色？
- (4) 把新节点染成红色可能违反哪条规则？
- (5) 如果新节点的父节点是黑色，插入后是否一定需要调整？

答案：

- (1) 原树为空时，新节点 a 应染成黑色，因为它是根节点，根必须为黑色。
- (2) 原树非空时，新节点通常先染成红色。
- (3) 若直接染黑，可能增加某些根到外部节点路径的黑高度，破坏 RB3。
- (4) 染成红色可能违反“红节点不能有红色孩子”的规则 RB2。
- (5) 如果新节点父节点是黑色，插入后通常不需要调整。

解析：

新节点先染红是为了尽量不改变黑高度。只有当父节点也是红色时，才产生连续红节点冲突并需要修复。

例题 18

题干：

在红黑树插入调整中，设：

u 表示新插入节点；

pu 表示 u 的父节点；

gu 表示 u 的祖父节点。

回答：

- (1) pu 是什么意思？
- (2) gu 是什么意思？
- (3) u 的叔节点是哪个节点？
- (4) 为什么红黑树插入调整主要关注 u、pu、gu 和叔节点？
- (5) 当 u 和 pu 都是红色时，违反了哪条红黑树性质？

答案：

- (1) pu 表示 u 的父节点。
- (2) gu 表示 u 的祖父节点。
- (3) u 的叔节点是 gu 的另一个孩子，也就是 pu 的兄弟节点。
- (4) 它们决定连续红冲突的局部结构以及应采用重新着色还是旋转。
- (5) 当 u 和 pu 都是红色时，违反 RB2。

解析：

红黑树插入修复是局部调整。父节点和叔节点颜色决定 XYr 或 XYb；祖父节点是旋转或重着色的局部中心。

例题 19

题干：

红黑树插入后出现连续两个红色节点。设 X 和 Y 表示从 gu 到 pu、再从 pu 到 u 的方向，X、Y 可以为 L 或 R；最后一个字母 r 或 b 表示叔节点颜色。

回答：

- (1) LLr 表示什么含义？
- (2) LRb 表示什么含义？
- (3) XYr 中的 r 指的是新插入节点颜色还是叔节点颜色？
- (4) XYb 中的 b 指的是新插入节点颜色还是叔节点颜色？
- (5) 为什么不能把 XYb 理解为“新插入节点是黑色”？

答案：

- (1) LLr 表示 pu 是 gu 的左孩子，u 是 pu 的左孩子，且叔节点为红色。
- (2) LRb 表示 pu 是 gu 的左孩子，u 是 pu 的右孩子，且叔节点为黑色。
- (3) XYr 中的 r 指叔节点颜色。
- (4) XYb 中的 b 也指叔节点颜色。
- (5) 因为新插入节点通常为红色，XYb 的 b 不是描述新节点，而是描述叔节点为 black。

解析：

X 和 Y 描述从祖父到父、从父到新节点的方向；最后的 r 或 b 描述叔节点颜色，这是判断调整方式的关键。

例题 20

题干：

红黑树插入后出现如下局部结构：

gu(黑)

/ \

pu(红) uncle(红)

/

u(红)

回答：

- (1) 该情况属于 XYr 还是 XYb？
- (2) 是否需要旋转？
- (3) pu 应改成什么颜色？
- (4) uncle 应改成什么颜色？
- (5) gu 应改成什么颜色？

(6) 如果 gu 是根节点，最后还要做什么处理？

答案：

- (1) 该情况属于 XYr，具体图示为 LLr。
- (2) 通常不需要旋转。
- (3) pu 改为黑色。
- (4) uncle 改为黑色。
- (5) gu 改为红色。
- (6) 如果 gu 是根节点，最后要把根节点重新染成黑色。

解析：

红叔情况通过重新着色即可消除局部连续红节点，同时保持经过父子两侧路径的黑高度一致。若祖父变红后继续向上冲突，则递归修复。

例题 21

题干：

红黑树插入后出现如下局部结构：

gu(黑)

/

pu(红)

\

u(红)

且 gu 的右孩子，即 u 的叔节点，为黑色外部节点。LRb

回答：

- (1) 该情况属于 LLb、LRb、RRb、RLb 中的哪一种？
- (2) 应采用重新着色还是旋转？
- (3) 应采用单旋还是双旋？
- (4) 旋转后哪个节点成为该局部子树的新根？
- (5) 旋转后新根应染成什么颜色？
- (6) 其他相关节点通常应染成什么颜色？

答案：

- (1) 该情况属于 LRb。
- (2) 应采用旋转并重新着色。
- (3) 应采用双旋。
- (4) 旋转后 u 成为该局部子树的新根。
- (5) 旋转后新根 u 应染成黑色。
- (6) 原 pu 和 gu 通常染成红色。

解析：

黑叔且方向为 LR 时是折线型，需要先把 pu-u 调整成直线，再让中间节点上升为局部根，并通过重新着色恢复红黑性质。

例题 22

题干：

从一棵空红黑树开始，依次插入：

50, 30, 70, 20

规则：

空树插入的根节点染黑；

非空树插入的新节点先染红；

若出现连续红色节点，再按红黑树插入规则调整。

回答：

- (1) 插入 50 后，50 是什么颜色？
- (2) 插入 30 后，是否违反 RB2？
- (3) 插入 70 后，是否违反 RB2？
- (4) 插入 20 后，u、pu、gu 分别是谁？
- (5) 插入 20 后，叔节点是谁？颜色是什么？
- (6) 该情况属于 XYr 还是 XYb？
- (7) 应如何调整颜色？
- (8) 调整后根节点最终是什么颜色？

答案：

- (1) 插入 50 后，50 为黑色。
- (2) 插入 30 后不违反 RB2，因为 30 为红色但父节点 50 为黑色。
- (3) 插入 70 后不违反 RB2，因为 70 的父节点 50 为黑色。
- (4) 插入 20 后，u=20，pu=30，gu=50。
- (5) 叔节点是 70，颜色为红色。
- (6) 该情况属于 XYr，具体为 LLr。
- (7) 调整为：30 和 70 染黑，50 染红；因为 50 是根，再把 50 染回黑色。
- (8) 调整后根节点最终为黑色。

解析：

插入 20 后出现 u 和 pu 连续红，但叔节点 70 也是红色，所以采用红叔重新着色，不需要旋转。根节点最后必须保持黑色。

例题 23

题干：

比较 AVL 树与红黑树。

回答：

LL LR RL RR

LLr LRr RLr RRr

LLb LRb RLb RRb

- (1) AVL 树使用什么信息判断节点是否平衡？
- (2) 红黑树使用什么规则维持近似平衡？
- (3) 哪一种树对高度平衡要求更严格？
- (4) 哪一种树通常旋转次数更少？
- (5) 哪一种树更适合频繁插入和删除的场景？
- (6) 哪一种树在结构上通常更矮、更适合查找密集场景？

答案：

- (1) AVL 树使用平衡因子 BF 或高度信息判断节点是否平衡。
- (2) 红黑树使用根黑、红节点无红孩子、所有根到外部节点路径黑高度相同等颜色规则维持近似平衡。
- (3) AVL 树对高度平衡要求更严格。
- (4) 红黑树通常旋转次数更少。
- (5) 红黑树更适合频繁插入和删除的场景。
- (6) AVL 树通常更矮，更适合查找密集场景。

解析：

AVL 用更严格的高度约束换取更短查找路径；红黑树用较宽松的颜色约束减少更新维护成本。

例题 24

题干：

某系统需要维护大量整数 key，并支持频繁查找、插入和删除。

回答：

- (1) 如果使用普通二叉搜索树，最坏情况下会出现什么问题？
- (2) AVL 树如何避免普通 BST 退化？
- (3) 红黑树如何避免普通 BST 退化？
- (4) AVL 树插入后主要根据什么判断是否旋转？
- (5) 红黑树插入后主要根据什么判断是重新着色还是旋转？
- (6) AVL 树和红黑树的查找、插入、删除时间复杂度通常都是多少？

答案：

- (1) 普通 BST 最坏情况下会退化成链表，查找、插入、删除可能变为 $O(n)$ 。
- (2) AVL 树通过维护 BF 在 -1、0、1 范围内，并在失衡时旋转来避免退化。
- (3) 红黑树通过颜色规则、重新着色和旋转限制树高。
- (4) AVL 插入后主要根据平衡因子及 LL、LR、RR、RL 类型判断是否旋转。
- (5) 红黑树插入后主要根据父节点、叔节点颜色以及 XYr/XYb 类型判断重新着色还是旋转。
- (6) AVL 树和红黑树的查找、插入、删除通常都是 $O(\log n)$ 。

解析：

平衡搜索树的共同目标是把树高限制在对数级。AVL 更严格，红黑树更灵活，但二者都能避免普通 BST 的线性退化。